

Разработка .Net приложения

Инструменты: Visual Studio 2022, MS SQL Server 2019/MySQL, Entity Framework Core, Serilog, HTML5, JavaScript, CSS/SCSS

Настройка:

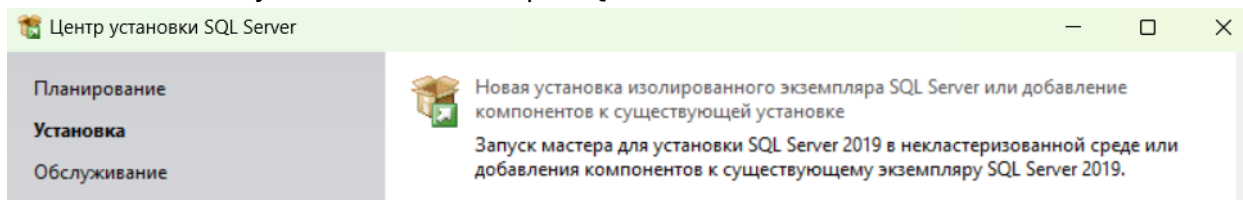
1. В PowerShell или командной строке проверяем версию: `dotnet --version`
2. Если её нет - устанавливаем. <https://dotnet.microsoft.com/ru-ru/download/dotnet/10.0>
Нужно установить пакет SDK и среду выполнения ASP.Net
3. Далее проверяем версию Entity Framework: `dotnet ef --version`
4. Если нет устанавливаем либо на компьютере(`dotnet tool install --global dotnet-ef`) либо

в проект. **В созданном проекте** открываем терминал и пишем `dotnet new tool-manifest`
`Dotnet tool install dotnet-ef`

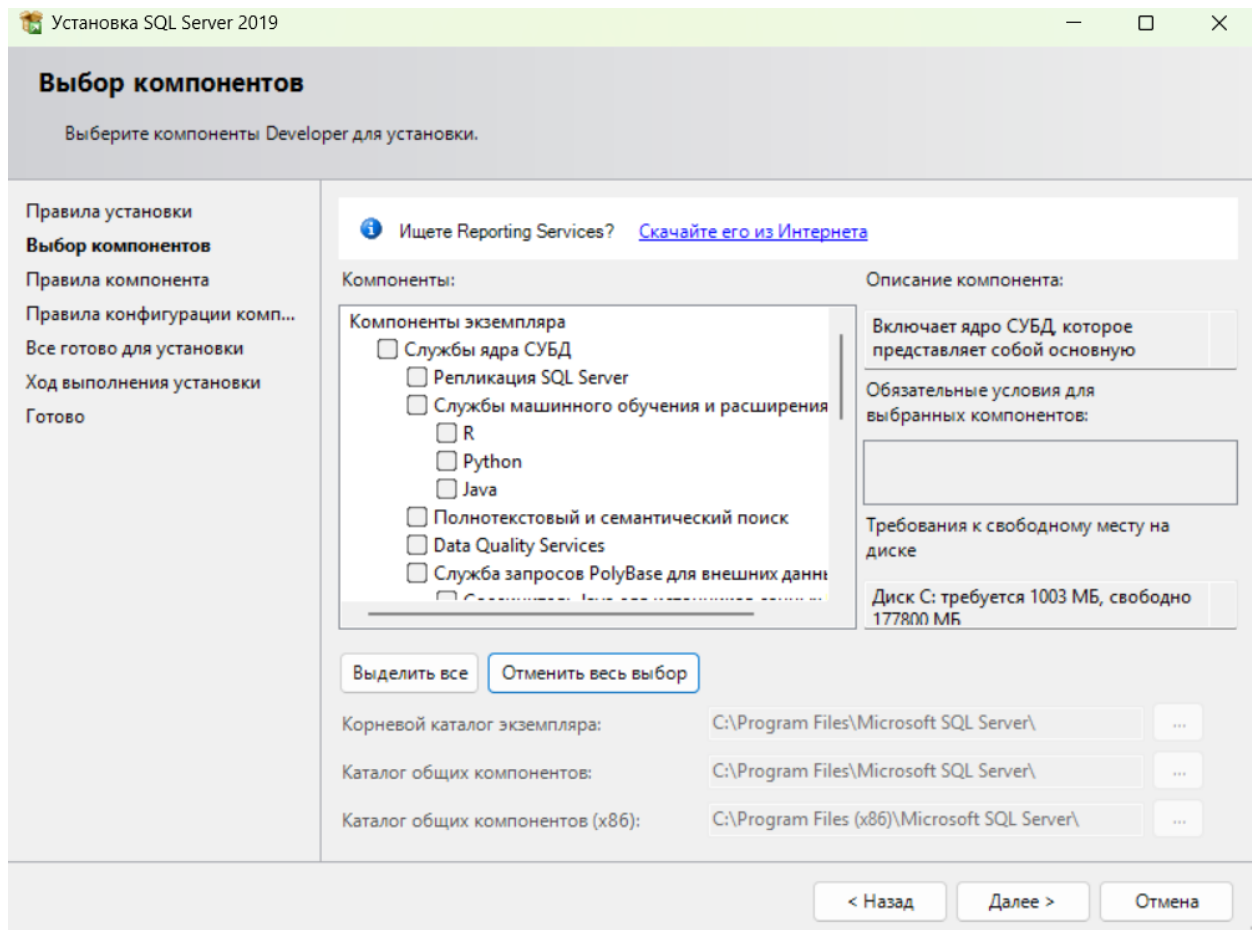
```
PS C:\Users\Аня\source\repos\WebApplication1> dotnet ef --version
Не удалось выполнить, поскольку указанная команда или файл не найдены.
Возможные причины этого включают:
* Вы допустили ошибку во встроенной команде dotnet.
* Вы собирались выполнить программу .NET, но dotnet-ef не существует.
* Вы собирались запустить глобальное средство, но в PATH не удалось найти исполняемый файл с префиксом dotnet и таким именем.
PS C:\Users\Аня\source\repos\WebApplication1> dotnet new tool-manifest
Шаблон "Файл манифеста локального инструмента dotnet" успешно создан.
PS C:\Users\Аня\source\repos\WebApplication1> dotnet tool install dotnet-ef
Вы можете вызвать средство из этого каталога с помощью следующих команд: "dotnet tool run dotnet-ef" или "dotnet dotnet-ef".
Средство "dotnet-ef" (версия "10.0.11") успешно установлено. В файл манифеста C:\Users\Аня\source\repos\WebApplication1\dotnet-tools.json добавлена запись.
PS C:\Users\Аня\source\repos\WebApplication1> dotnet ef --version
Entity Framework Core .NET Command-line Tools
10.0.11
```

Установка MS SQL Server (ссылка на видео как скачать SQL Server 2019 Developer Edition +
SQL Server Management Studio: https://youtu.be/dP_ZmYhNFIg?si=3worgOgdHKGooGk)

1. SQL Server 2019 Developer Edition: <https://go.microsoft.com/fwlink/?linkid=866662>
2. SQL Server Management Studio (все языки на ваш выбор)
<https://learn.microsoft.com/en-us/ssms/release-notes-20>
3. В SQL Server 2019 Developer Edition выбираем **Пользовательский**
4. Первый пункт Планирование можно пропустить нажимаем следующий пункт
Установка
5. Нажимаем Новая установка экземпляра SQL Server

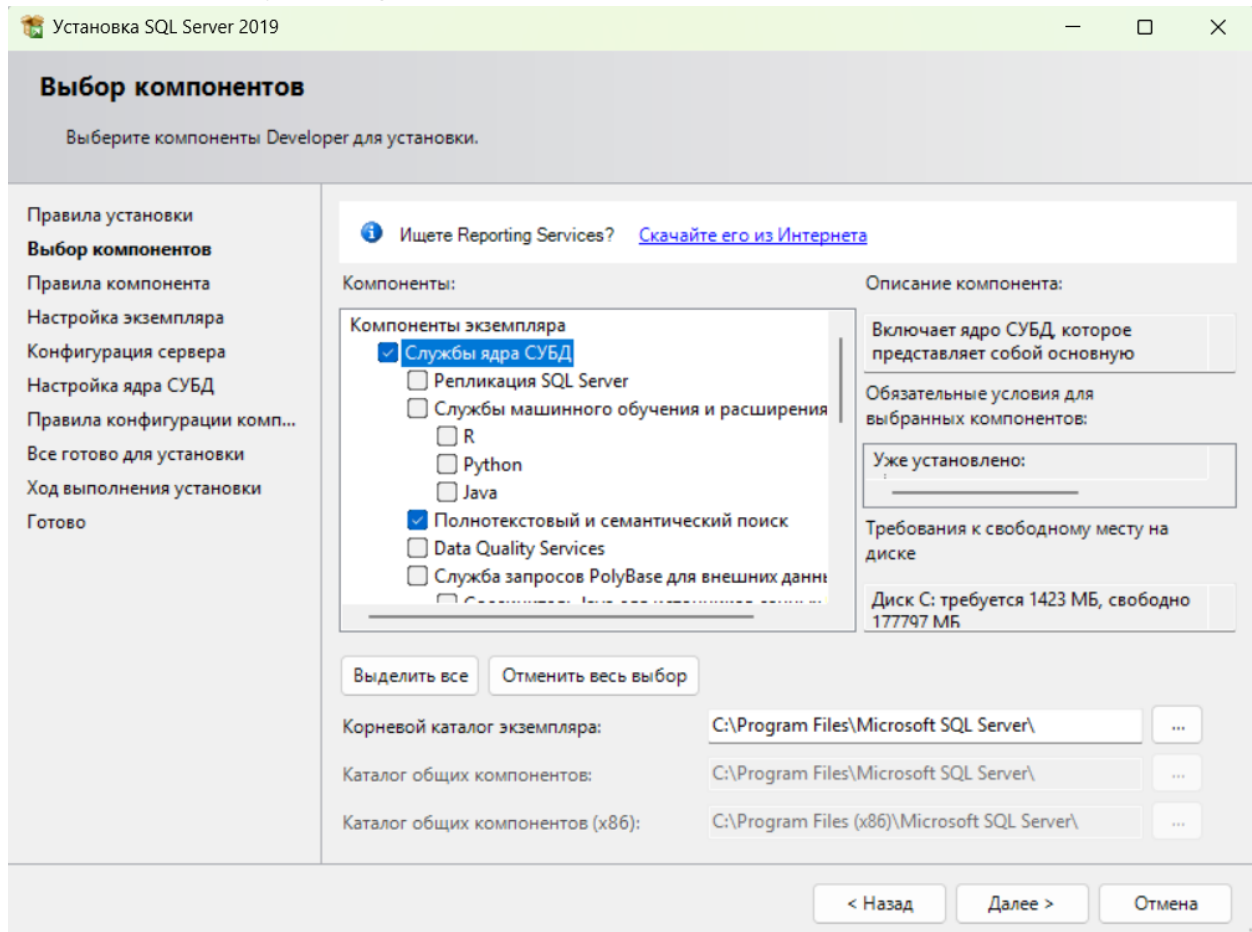


- 6.
7. Нажимаем Далее и доходим до Выбора компонентов. Нажимаем Отменить весь выбор



8.

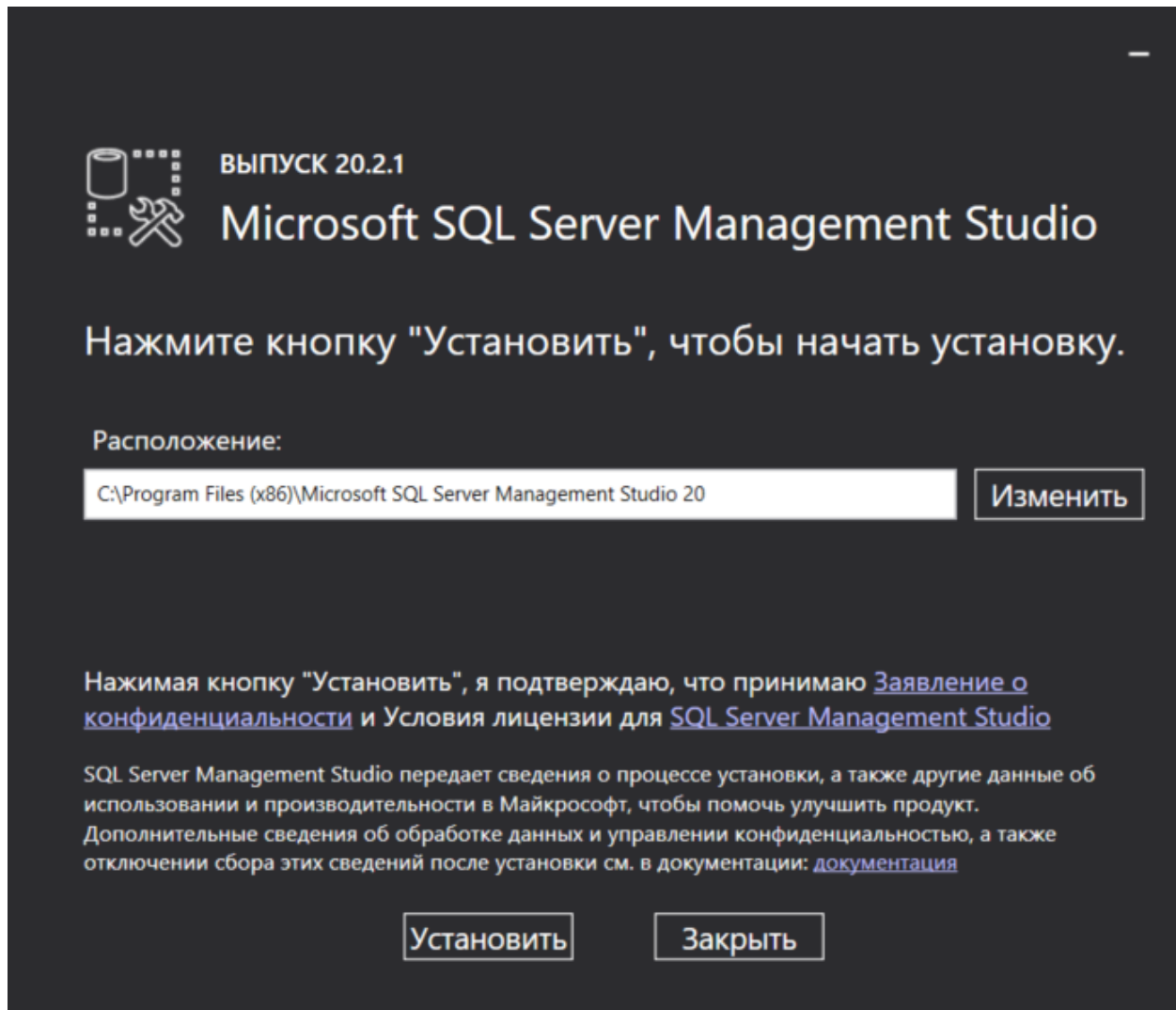
9. Устанавливаем Службы ядра СУБД и Полнотекстовой и семантический поиск



10.

11. Нажимаем Далее и доходим до Настройка ядра СУБД. Там нажимаем на кнопку Добавить текущего пользователя, нажимаем Далее и Установить.
12. Установка завершена

Установка SQL Server Management Studio



- 1.
2. Нажимаем установить

Запуск

1. Открываем Microsoft SQL Server Manegement Studio

Соединение с сервером

SQL Server

Имя для входа | Свойства соединения | Always Encrypted | Дополнительные параметры

Сервер

Тип сервера: Ядро СУБД

Имя сервера: DESKTOP-544HK4A

Проверка подлинности: Проверка подлинности Windows

Имя пользователя: DESKTOP-544HK4A\Аня

Пароль:

☐ Запомнить пароль

Безопасность подключения

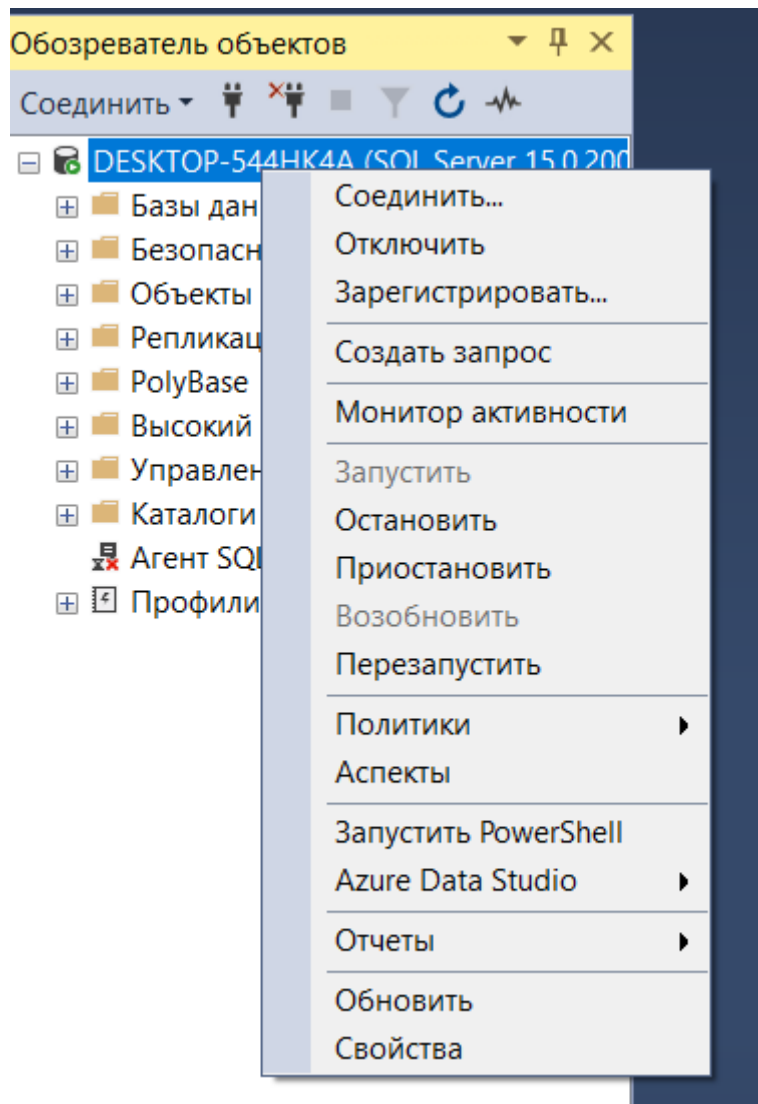
Шифрование: Обязательно

☐ Доверять сертификату сервера

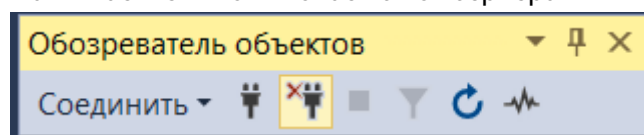
Имя узла в сертификате:

Соединить | Отмена | Справка | Параметры <<

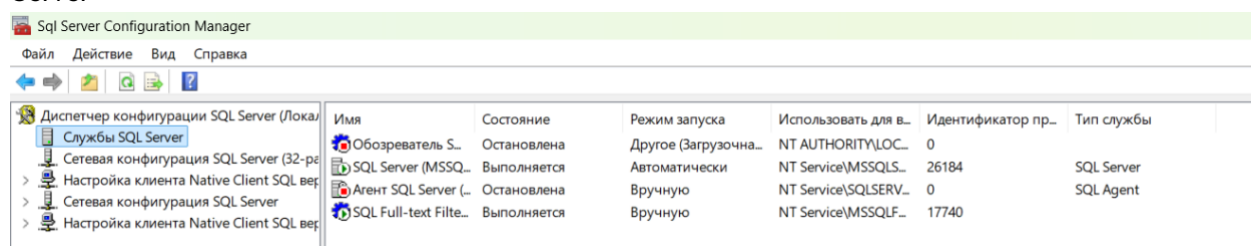
- 2.
3. Нажимаем Доверять сертификату сервера и шифрование необязательно
4. Подключаемся
5. Правой кнопкой мыши нажимаем на имя сервера и выбираем свойства



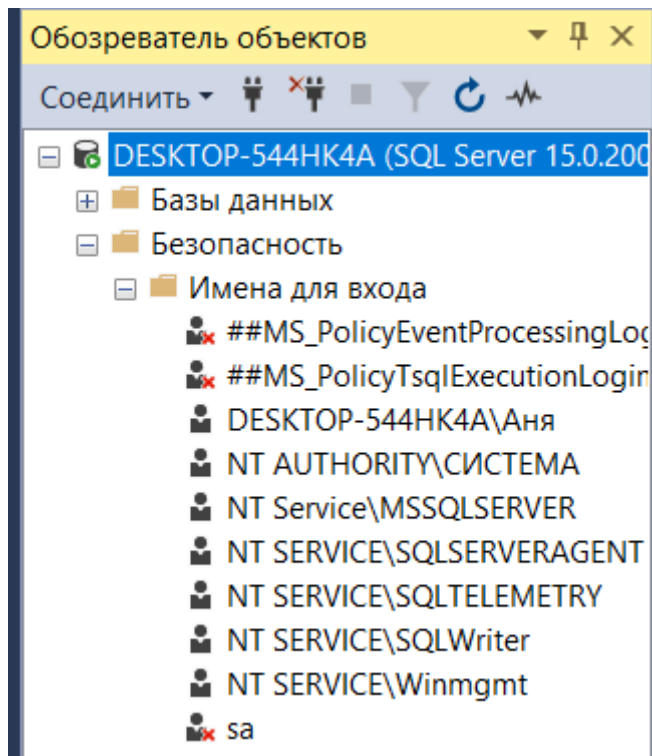
- 6.
7. Переходим во вкладку безопасность и выбираем Проверка подлинности SQL Server и Microsoft
8. Нажимаем ОК и отключаемся от сервера



- 9.
10. Заходим в SQL Server 2019 Configuration Manager. Заходим во вкладку Службы SQL Server



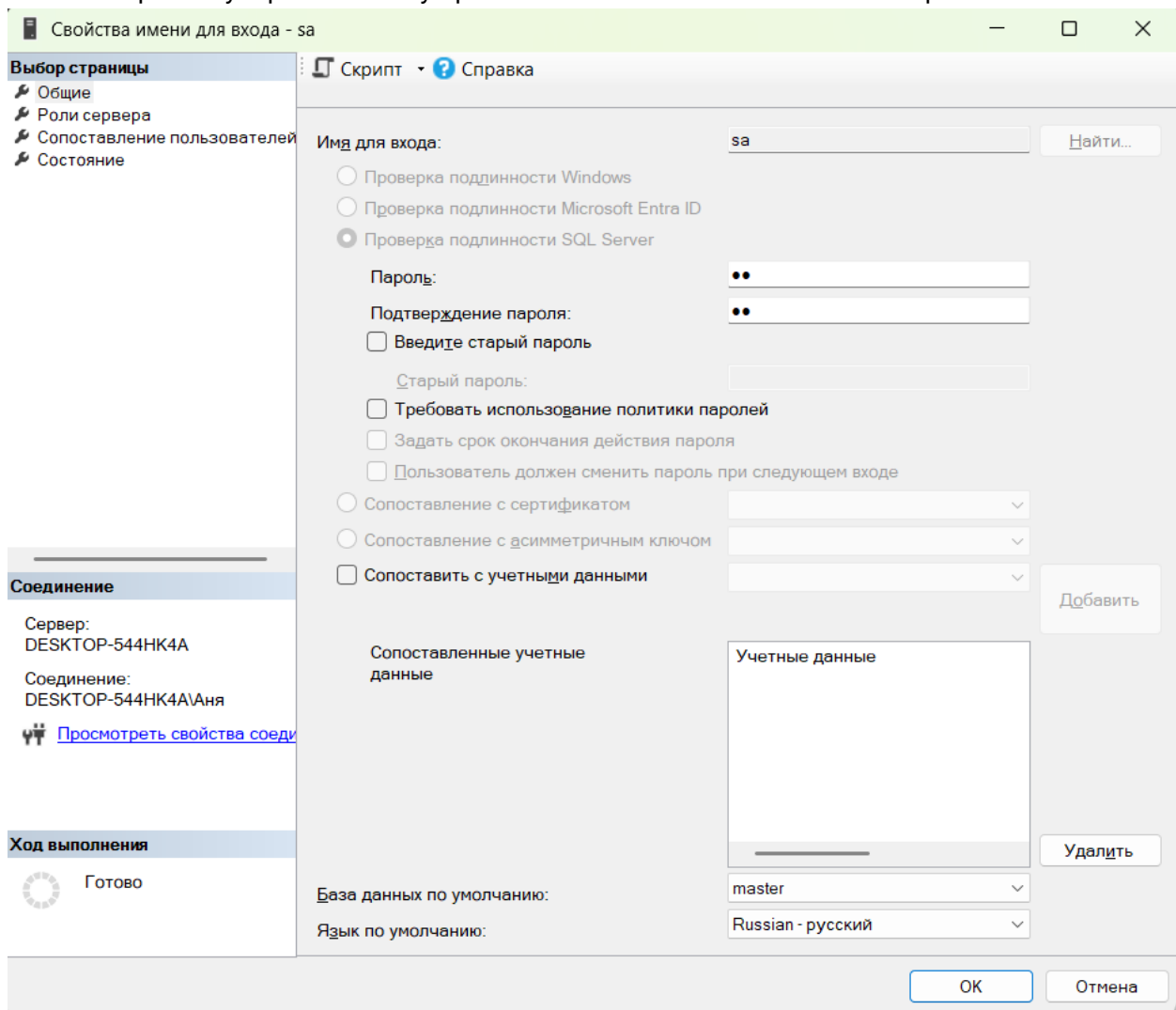
- 11.
12. Правой кнопкой мыши нажимаем на SQL Server и перезапускаем его
13. Снова подключаемся, переходим во вкладку безопасность -> имена для входа



14.

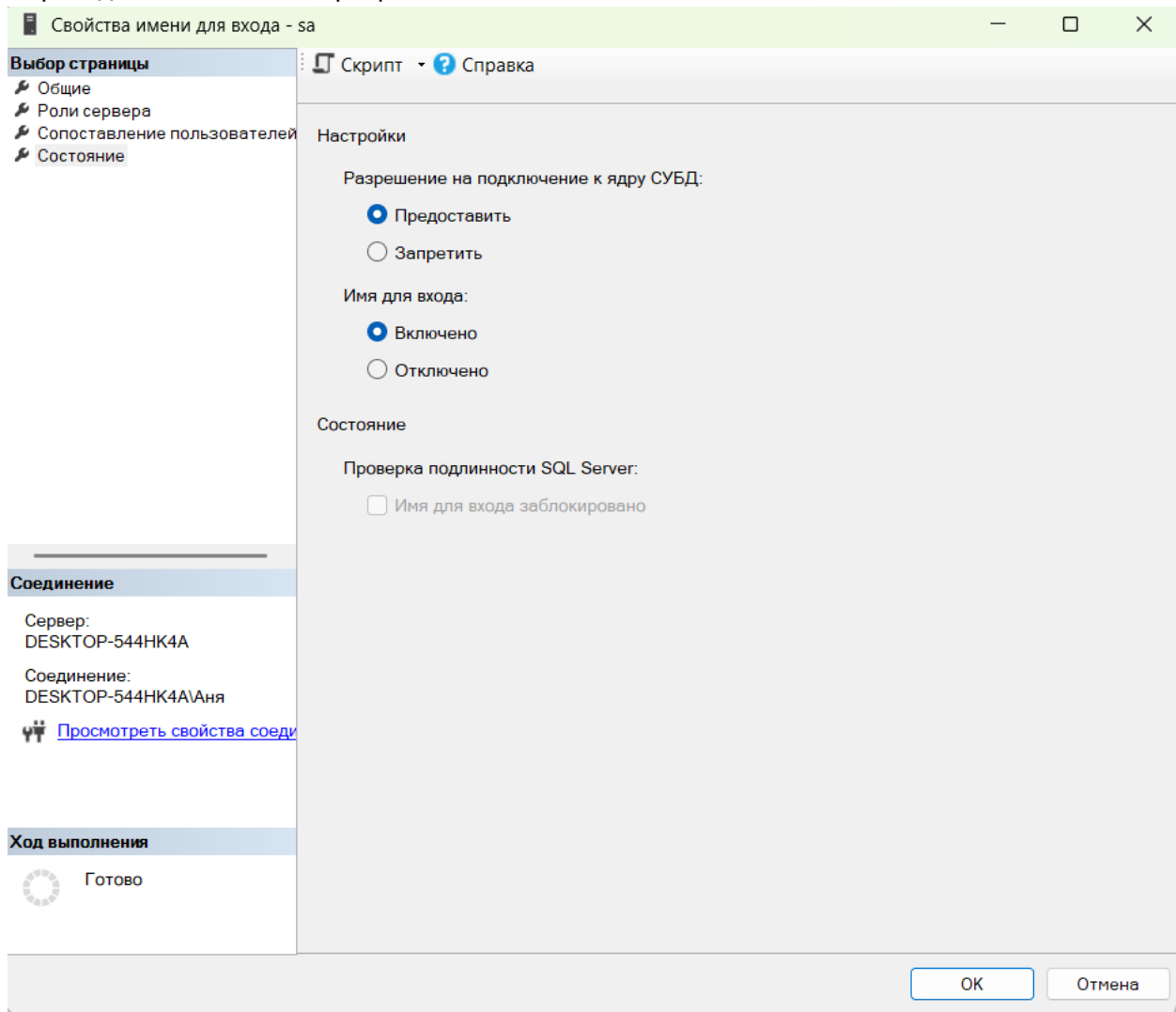
15. Открываем sa

16. Ставим пароль и убираем галочку Требовать использование политики пароля



17.

18. Переходим в Состояние и разрешаем логиниться



19.

20. Отключаемся, подключаемся и пробуем зайти под sa

Создание проекта

1. Открываем VS 2022 и выбираем Пустой шаблон [ASP.Net](#) Core
2. Выбираем имя, директорию, нажимаем далее и ставим галочки

Дополнительные сведения

Пустой шаблон ASP.NET Core (Майкрософт) C# Linux macOS Windows Облако Служба Веб

Платформа ⓘ
.NET 10.0 (долгосрочная поддержка)

☒ Настроить для HTTPS ⓘ
☐ Включить поддержку контейнера ⓘ

ОС контейнера ⓘ
Linux

Тип сборки контейнера ⓘ
Dockerfile

☒ Не использовать операторы верхнего уровня ⓘ
☐ Используйте домен верхнего уровня .dev.localhost в URL-адресе приложения ⓘ
☐ Включить в список проект оркестрации Aspire ⓘ

Версия Aspire ⓘ
13.5

- 3.
4. Создаем проект
5. Меняем сервер с http на IIS Express
6. Дважды щелкаем по проекту
7. В PropertyGroup добавляем `<TreatWarningsAsErrors>true</TreatWarningsAsErrors>`
8. Нажимаем правой кнопкой мыши по проекту и выбираем Управление пакетами NuGet
9. Устанавливаем Serilog.AspNetCore, Microsoft.AspNetCore.Identity.EntityFrameworkCore, Microsoft.EntityFrameworkCore.Design, Microsoft.EntityFrameworkCore.SqlServer

Макет

1. Готовый шаблон можно найти на сайте html5up.net
2. Заходим на tinytmc, создаем аккаунт. Во вкладке Integrate TinyMCE копируем API key
3. В файле appsettings.json убираем всё и прописываем настройки

```
{
  "Project": {
    "TinyMCE": {
      "APIKey": ".....",
    },
    "Company": {
      "CompanyName": "Облачко",
      "CompanyPhone": "+7 (111) 111-11-11",
      "CompanyPhoneShort": "+7111111111",
      "CompanyEmail": "contact@mycompany.gmail"
    }
  }
}
```
4. Создаем папку Infrastructure и в ней новый класс AppConfig

```
public class AppConfig
{
    public TinyMCE tinyMCE { get; set; } = new TinyMCE();
    public Company company { get; set; } = new Company();
}
```



```

public class TinyMCE
{
    public string? APIKey { get; set; }
}

public class Company
{
    public string? CompanyName { get; set; }
    public string? CompanyPhone { get; set; }
    public string? CompanyPhoneShort { get; set; }
    public string? CompanyEmail { get; set; }
}

```

5. Создаем структуру проекта

- wwwroot
 - css
 - img
 - js
 - scss
- Models
 - ViewComponents
- Views
 - Shared
- Controllers
- Domain
 - Entities
 - Enums
 - Repositories
 - Abstract
 - EntityFramework

Работа с html

1. Переносим файлы (кроме html) из скаченного макета в проект
2. Открываем страницу html в VS 2022
3. В папке Shares создаем файл _Layout.cshtml и переносим html.
4. Создаём заготовки частичных представлений
5. В той же папке создаем файл _CssPartical.cshtml и помещаем туда подключение к css
6. Вместо assets ставим ~
7. Тоже самое делаем с js
8. В файле с html на место вырезаного кода вставляем @await Html.PartialAsync("_Название файла")
9. Дальше разбираем код (отдельные файлы для header, footer и тд)
10. В файле где находятся мета теги внедряем конфигурицию @inject IConfiguration config
11. В папке Views создаем два файла _ViewImports.cshtml и _ViewStart.cshtml
12. В ViewStar пишем


```
@{
    Layout = "_Layout";
}
```
13. В _ViewImports @addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers


```
@using Microsoft.AspNetCore.Mvc.TagHelpers
@using WebApplication1.Infrastructure
```
14. В мета тегах добавляем

```
@{
    AppConfig? appConfig = config.GetSection("Project").Get<AppConfig>();
}
```

```
<title>@(ViewBag.Title ?? appConfig?.company.CompanyName)</title>
```

15. В _FooterPartial.cshtml прописываем подключение к конфигу и в меню пишем название компании в шапке делаем тоже самое

16. Убираем меню из header. Не забываем подключать в Layout

17.

```
1 @inject IConfiguration config
2 @{
3     AppConfig? appConfig = config.GetSection("Project").Get<AppConfig>();
4 }
5 <div id="header-wrapper">
6     <div class="container">
7         <div id="header">
8             <div class="inner">
9                 <h1><a asp-area="" asp-controller="Home" asp-action="Index" id="logo">@appConfig?.company.CompanyName</a></h1>
10            </div>
11        </div>
12    </div>
13 </div>
14 </div>
```

18.

```
1 @inject IConfiguration config
2 @{
3     AppConfig? appConfig = config.GetSection("Project").Get<AppConfig>();
4 }
5 <div id="footer-wrapper">
6     <div id="footer" class="container">
7         <div class="row">
8             <div class="col-12">
9                 <div id="copyright">
10                     <ul class="menu">
11                         <li>@appConfig?.company.CompanyName</li>
12                         <li>Design: <a href="http://html5up.net">HTML5 UP</a></li>
13                     </ul>
14                 </div>
15             </div>
16         </div>
17     </div>
18 </div>
```

Атрибут asp-controller назначает контроллер, используемый для создания URL-адреса

Значение атрибута asp-action представляет имя действия контроллера, включенное в созданный атрибут href

Атрибут asp-area указывает имя области, используемое платформой для определения соответствующего маршрута.

Создание контроллера

1. Создаем в папке Controllers пустой контроллер MVC
2. Нажимаем правой кнопкой мыши по Index и выбираем добавить представление

3. Выбираем пустое представление (Название представление должно соответствовать названию действия)
4. Из Layout вырезаем контент и вставляем в представление
5. На место вырезаного контента пишем @RenderBody()

Работа в Program.cs

1. Делаем метод Main асинхронным и вместо void пишем Task
2. Вместо var объявляем явный тип
3. Подключаем в конфигурацию файл appsettings.json
4. Оборачиваем секцию Project в объектную форму
5. Подключаем функционал контроллеров
6. Собираем приложение
7. Подключаем использование статичных файлов
8. Подключаем систему маршрутизации
9. Регистрируем маршруты

10.

Программирование доменной модели

1. В папке enums создаем класс ServiceTypeEnum
2. Меняем class на enum
3. Записываем нужное перечисление

```

1 namespace WebApplication1.Domain.Enums
2 {
3     Ссылка: 0
4     public enum ServiceTypeEnum
5     {
6         Ссылка: 0
7         Standard,
8         Ссылка: 0
9         VIP
10    }
11 }

```

4.

5. В папке Entities создаем класс EntityBase и заполняем его

```

1 using System.ComponentModel;
2 using System.ComponentModel.DataAnnotations;
3
4 namespace WebApplication1.Domain.Entities
5 {
6     Ссылка: 0
7     public abstract class EntityBase
8     {
9         Ссылка: 0
10        public int Id { get; set; }
11
12        [Required(ErrorMessage = "Заполните название")]
13        [Display(Name = "Название")]
14        [MaxLength(200)]
15        Ссылка: 0
16        public string? title { get; set; }
17
18        Ссылка: 0
19        public DateTime DateCreated { get; set; } = DateTime.UtcNow;
20    }
21 }

```

6.

7. Создаем нужные вам сущности. Например категория и услуги

```

1 namespace WebApplication1.Domain.Entities
2 {
3     Ссылка: 0
4     public class ServiceCategory : EntityBase
5     {
6         Ссылка: 0
7         public ICollection<Service> Services { get; set; }
8     }
9 }

```

8.

```
AppDbContext.cs | Service.cs* | ServiceCategory.cs* | EntityBase.cs* | ServiceTypeEnum.cs*
WebApplication1 | WebApplication1.Domain.Entities.Service

1  using System.ComponentModel.DataAnnotations;
2  using WebApplication1.Domain.Enums;
3
4  namespace WebApplication1.Domain.Entities
5  {
6      Ссылка: 1
7      public class Service : EntityBase
8      {
9          [Display(Name = "Выберите категорию, к которой относится услуга")]
10         Ссылка: 0
11         public int? ServiceCategoryId { get; set; }
12         Ссылка: 0
13         public ServiceCategory? ServiceCategory { get; set; }
14
15         [Display(Name = "Краткое описание")]
16         [MaxLength(3_000)]
17         Ссылка: 0
18         public string? DescriptionShort { get; set; }
19
20         [Display(Name = "Описание")]
21         [MaxLength(100_000)]
22         Ссылка: 0
23         public string? Description { get; set; }
24
25         [Display(Name = "Титульная картинка")]
26         Ссылка: 0
27         public string? Photo { get; set; }
28
29         [Display(Name = "Тип услуги")]
30         Ссылка: 0
31         public ServiceTypeEnum Type { get; set; }
32     }
33 }
```

9.

Создаем контекст базы данных

1. В папке Domain создаем файл AppDbContext
2. Прописываем базовые настройки

```

AppDbContext.cs*  Service.cs  ServiceCategory.cs  EntityBase.cs  ServiceTypeEnum.cs
WebApplication1  WebApplication1.Domain.AppDbContext

1  using Microsoft.AspNetCore.Identity;
2  using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
3  using Microsoft.EntityFrameworkCore;
4  using WebApplication1.Domain.Entities;
5
6  namespace WebApplication1.Domain
7  {
8      Ссылка: 2
9      public class AppDbContext : IdentityDbContext<IdentityUser>
10     {
11         Ссылка: 0
12         public DbSet<ServiceCategory> ServiceCategories { get; set; } = null!;
13         Ссылка: 0
14         public DbSet<Service> Services { get; set; } = null!;
15
16         Ссылка: 0
17         public AppDbContext(DbContextOptions<AppDbContext> options): base(options)
18         {
19         }
20
21         Ссылка: 0
22         protected override void OnModelCreating(ModelBuilder builder)
23         {
24             base.OnModelCreating(builder);
25
26             string adminName = "admin";
27             string roleAdminId = "DA36513F-6393-43B8-A1E1-5CDF3D8EB6BF";
28         }
29     }
30 }

```

- 3.
4. Для расширенных настроек нужно создать GUID
5. Заходим в Средства -> Создать GUID. Выбираем 4. Формат реестра и вставляем в roleAdminId и userAdminId
6. Добавляем роль администратора сайта

```

Ссылка: 0
protected override void OnModelCreating(ModelBuilder builder)
{
    base.OnModelCreating(builder);

    string adminName = "admin";
    string roleAdminId = "DA36513F-6393-43B8-A1E1-5CDF3D8EB6BF";
    string userAdminId = "FA2325ED-7786-447A-BA83-93C89FE17D93";

    builder.Entity<IdentityRole>().HasData(new IdentityRole()
    {
        Id = roleAdminId,
        Name = adminName,
        NormalizedName = adminName.ToUpper(),
    });
}
}

```

- 7.
8. Так же добавляем пользователя в качестве администратора сайта

```

});

builder.Entity<IdentityUser>().HasData(new IdentityUser()
{
    Id = userAdminId,
    UserName = adminName,
    NormalizedUserName = adminName.ToUpper(),
    Email = "admin@admin.com",
    NormalizedEmail = "admin@admin.com",|
    EmailConfirmed = true,
    PasswordHash = new PasswordHasher<IdentityUser>().HashPassword(new IdentityUser(), adminName),
    SecurityStamp = string.Empty,
    PhoneNumberConfirmed = true
});
}

```

9.

10. Определяем фдмина в соответствующую роль

```

});

builder.Entity<IdentityUserRole<string>>().HasData(new IdentityUserRole<string>()
{
    RoleId = roleAdminId,
    UserId = userAdminId,
});
}

```

11.